

抽卡模拟器

0.背景，是不必要的！

小w的朋友(绝对不是无中生有!)最近玩游戏抽卡有点抽破防了，出于人道主义精神，小w决定为他量身设计一款抽卡模拟器，为他提供一个免费 破防 发泄的平台。至于实现嘛，小w将如此重任交予了你！



1.规范，是首要的！

- 如果你有使用python/c/c++的需求，也不是不行，不过还是建议使用 **golang** 完成该项目
- 先了解一下 **git commit规范**，将代码上传到github仓库(请使用public)。
- 恰用git，每完成一个功能注意commit
- 注意项目文件的架构，了解并尽量遵循 **MVC** 格式
- 关于api的设计，建议先看看 **RESTful API** 规范
- 注意整理 **dev log**，记录自己的开发与心路历程或者一些你想说的
- 题目内容可能有点大，有些需求相对比较独立，如果遇到困难，可以暂时放置，不必全部完成，尽力而为即可
- 你可以先浏览一下整个流程再进行开发
- 题目中 **额外功能** 不做强制要求，你可以在有时间的情况下再完成

2.总要先有个账号吧

毕竟还要统计信息，我们还是有一个账号比较好。

我们的账号系统有以下简单的要求：

- 实现用户的注册、登录、修改信息、登出；注册时生成一串id用于id+password登录
- 用户信息：昵称、id、密码、简介、一些你觉得会用上的字段
- 提供一个admin账号，该角色的权限和普通用户有所区别，将在之后的任务中体现

实现以上需求，你需要挑选一个合适的数据库存储数据。关于数据库的操作，你可以选择相应的ORM框架。

身份鉴权方式？可以看看 **JWT**

额外功能：

这种登录太简单粗暴了？可以尝试接入邮箱，添加邮箱+验证码登录方式，相应逻辑日常还挺常见的吧()

3.开抽！

我们先考虑一个角色池子：

- 角色一共分为 S A B 三档，其中S档角色分为限定和非限定以及up和非up，非限定角色一直存在于池子中，限定角色会被移出池子
- 出现S档角色时，有50%的概率为up角色，剩下50%概率由当前池子限定和非限定角色均分；出现A/B档角色时，从所有相应档位角色中随机一位
- 角色的基本属性有：档位，名称，是否限定，你认为还需要的属性

众所周知，抽卡是有一定机制的，我们的机制规定如下：(概率有点眼熟？不要在意这些细节)

- 每档角色有相应的概率，S档基础概率 **0.8000%**，A档 **8.0000%**，B档 **91.2000%**
- 一个池子里的限定S档角色上限为 **3**
- 每 **10次** 抽取，必定有一位A档角色，即如果第10次抽取仍没有A档，强制改为A档角色；如果提前出现A档角色，从当前抽开始重置计数
- 每 **80次** 抽取，从第 **65** 抽开始，每次抽取S档概率追加 **5.0000%**，A档概率不调整，即第65抽S档概率为0.8000%，第66抽为5.8000%，以此类推；每80抽必定有一位S档角色，即如果第80次抽取仍没有S档，强制改为S档角色；如果提前出现S档角色，从当前抽开始重置计数和概率。

设计的时候，这些数据不建议写死

用户端可以进行的操作有：

- 一次性抽一发 (可选)
- 一次性抽十发
- 查看当前卡池概率信息和角色信息(主要是获取当前up的S档和限定的S档角色)

- 查看抽卡记录，要求可以查看每次抽到的角色档位与基本信息(可以只是名称)，以及是否是第一次抽到
- 查看拥有的角色，要求可以看到拥有角色的等阶(即潜能/命座/etc. 抽到过几次，首次抽到等阶为0)

admin端可以进行的操作：

- 添加新的角色数据(注意，比如添加了一个B档角色后，池子里立马就能抽到，可能会有点不合理)
- 推送角色入池：
 - 指明档位、是否限定、是否up (后两项只需S档角色考虑即可)
 - S档：如果限定+当前池子限定数达到上限，自动将最早入池的角色移出；如果是up，进行调整保证池子里只有一位up

注意，以上有些接口可能会涉及到多次的数据库操作，众所周知，一致性是很重要的，可以了解一下数据库的**事务**

额外功能

1. 你可能会见过这种抽卡数据统计，咱的抽卡器也不能落下这么 ~~诛心~~ 好玩的功能嘛！为此，请你为用户再提供一个接口，提供每个S档角色获取到时抽取的次数 (歪没歪这个标签呢？就看你想不想实现了)



2. 不能让非酋污染我的抽卡记录！为此，我们再实现一个清空历史记录接口，让用户可以删除自己的抽卡记录，当然，拥有的角色也要删掉的说

4.我要偷改概率！

注意：stage 4和stage 5的任务较为灵活，你可以尽力而为，可以不用全部实现

4.1 好像需要解耦一点

这里，你可能要对先前完成的工作进行一点改动

我们现在需要把上面的功能拆分成两部分：**Management Server** 和 **Game Server**

其中，Management Server 来负责上述的账号系统功能与admin的操作(即添加角色与角色入池)，Game Server 来负责抽卡逻辑工作。

ps: 如果是jwt的话，这个时候game server怎么解析token呢？复制一遍key吗，还有什么方法吗，思考一下

4.2 要能改概率才对嘛

在此之前，你需要先去简单了解一下RPC

什么，非酋？达咩！为此，我们要给admin一个修改概率的功能

回到stage 3中的概率，我们这个接口希望可以修改其中 `xxx` 标记的那么几个数据，这个接口希望设计在**Management Server** 中。那么，修改数据后，**Game Server** 怎么感知并进行热重载使用新数据呢？

我们决定用 **gRPC** 来实现这个功能。具体来说：

Management Server:

- 提供一个面向用户的api接口，实现修改卡池概率逻辑，当然，这个接口admin才有权限
- 提供一个面向其他内部服务的grpc接口，采用 **Server-side Streaming(服务端流式传输)** 模式，并维护所有订阅连接
- 当api接口收到修改请求后，同步将变更后的数据写入 grpc 流中，推送给所有订阅者

(tips: 这里或许会用上管道？)

Game Server:

- 作为客户端调用 `Management Server` 的grpc接口，并保持长连接监听。
- 持续读取流中的变更数据，当捕获到新配置时对数据进行修改，确保后续的抽卡请求使用上新概率。
- 这时候注意一下怎么处理并发问题？注意不要影响到正在发生的抽卡

tips:(可能的一种处理方式)如果抽卡的时候使用的概率参数是一份快照呢

grpc的服务端客户端代码可参考 <https://github.com/grpc/grpc-go/tree/master/examples/helloworld> (golang)

5.还是通知一下吧()

偷偷改概率什么的好像是有点不地道了哈。那么我们就给用户一点小通知吧！

- 在admin调用修改概率后，`Management Server` 再向客户端发送概率修改的通知。只需要向在线的用户投放即可，不用考虑其他未建立连接的用户。

事已至此，我们就再多通知一点吧()

- 为之前实现的admin的推送角色入池接口也加上这个功能，每当池子有变动时通知用户

这里可以考虑使用 **SSE**

6.万事俱备，只欠套壳!

至此，我们的抽卡模拟器的功能基本上完善了，不过要怎么展示呢？首先，我们需要：

- 准备一个api文档。你可以采用你认为舒服的方式，如md文档、swagger、apifox(这个用网页版就好)

展示方面，你可以选择vibe或者简单搓一个cli客户端，包装一下你的接口以方便调用；或者vibe一个前端页面

END

恭喜，咱的抽卡模拟器堂堂诞生啦！接下来，你可以：完善前面未实现的功能、添加一些你觉得好玩的新功能，梳理梳理整个流程和思路。你说，小w的朋友会喜欢上这个模拟器吗()

